



VIT[®]
B H O P A L
www.vitbhopal.ac.in

*School of Computing Science and Artificial
Intelligence (SCAI)*

Name: Shweta Kumari

Reg. no.:25BAI11470

Branch: CSE AI & ML

Subject: Programming in Java

Slot: B14+D21



VIT[®]
B H O P A L
www.vitbhopal.ac.in

- VIT Bhopal - First Private University in India to have 100% Doctoral Faculty
- VIT Bhopal - First University in India to introduce CALTech - Collaborative and Active Learning through Technology

Anchor – Your Life Tracking Buddy

Abstract

Students these days juggle a lot of things at once — studying, assignments, money, and trying to build good habits. And for each of those, there's usually a separate app. A Pomodoro timer here, a to-do app there, a spending tracker, a habit app, and so on. It works, but only up to a point. The problem is that none of these apps know about each other. You can see how much you studied this week, but you can't easily tell whether that was a good week overall. There's no single place that ties everything together.

Anchor is my attempt to fix that. It's a Windows desktop application that combines a study timer, a to-do list, an expense tracker, and a habit tracker into one program. All four use the same datebased database, which means the dashboard can pull data from all of them at once and show a combined view of the day. There's even a "consistency score" that tells you, on a scale of 0 to 100, how on-track you were.

The app is built in Java 17 with JavaFX for the interface and SQLite for local storage. It's fully offline — no accounts, no cloud, no internet required. Once you install it, everything stays on your machine. The whole thing ships as a single Windows installer built with the JDK's jpackage tool, so anyone can install and run it without needing Java installed first.

Table of Contents

1. Introduction
2. Existing Tools and Their Gaps
3. Problem Statement
4. Objectives
5. System Requirements
6. Design
7. Implementation
8. Testing
9. Screenshots
10. Conclusion
11. Future Work
12. References

1. Introduction Background

Anyone who's tried to be a productive student knows the drill. You open a Pomodoro timer to study, then switch to a to-do app to plan the day, then check your expense tracker to see how much you've spent, and finally tick off habits in yet another app. Four tools, four separate logins, four separate data silos. It's exhausting, and honestly, most people give up on at least one of them within a few weeks.

The frustrating part isn't that any individual app is bad. It's that they never talk to each other. I can look at my study app and see I studied for three hours yesterday. But I can't look at anything and see whether yesterday was actually a good day overall. Did I also finish my tasks? Did I stay under budget? Did I keep my habits going?

Why I Built This

I'll be honest about where this project came from. I was using three or four separate apps myself, and the switching back and forth was annoying enough that I kept skipping days. Not because I didn't want to log things, but because it was too much friction. Then I started thinking: what if I just built one tool that does all of it, and shows me a single picture?

That's Anchor. It's not trying to compete with Notion or anything enterprise. It's a small, focused personal tool that does four things well and one thing better than any of the others could individually — showing me how consistent I actually am.

Scope

This project is a Windows-only desktop app for a single user (me). I intentionally kept it small: no accounts, no cloud sync, no mobile version. All data lives locally in a SQLite file. That decision was partly practical — I didn't want to build user management for a personal tool — and partly philosophical — the whole point is to reduce friction, not add login screens.

2. Existing Tools and Their Gaps

Before building Anchor, I looked at what's already out there. There are good tools in each category:

The pattern is clear: each tool is good at its own thing, but none of them connect. There's no lightweight, offline, native Windows tool that combines these four areas and surfaces a single consistency view.

That's the gap Anchor fills.

Tool	What it does	Where it falls short
Pomofocus, Focus To-Do	Pomodoro timers	Only track study time
Todoist, Microsoft To-Do	Task management	Nothing about money or habits
Walnut, Money Manager	Expense tracking	No link to study or tasks
Habitica, Loop	Habit tracking	No unified dashboard
Notion	Generic productivity	Heavy, online-dependent, takes effort to set up

3. Problem Statement

Students use 3–4 separate applications to manage their daily routine. This causes three specific problems:

No unified view. Each app shows its own slice, but nothing shows the whole picture. You can't answer "was this a good week?" without manually checking four different places.

No cross-module insights. Because the data is scattered, you can't spot patterns like "I always skip studying on days I overspend" or "my habit streak correlates with good study weeks."

Friction kills consistency. Logging something in four apps every day is a chore. Eventually you skip a day, then a week, then you stop altogether. The tools that were supposed to help end up abandoned.

4. Objectives

The concrete goals for this project were:

1. Build one Windows app that replaces four separate tools.
 2. Make logging fast — ideally 2–3 clicks for anything.
-

3. Show trends and a consistency view, not just raw logs.
4. Make it pleasant enough that I'd actually want to open it daily.
5. Ship it as a native Windows installer that doesn't need Java installed.
6. Keep everything offline and local — no accounts, no cloud.

5. System Requirements

Hardware

- Any Windows machine from the last decade
- 4 GB RAM minimum, 8 GB recommended
- 500 MB free disk space
- 1024×768 minimum screen resolution

Software

- Windows 10 or Windows 11 (64-bit)
- No separate Java install required at runtime — the installer bundles everything

Development Tools I Used

What	Version
Java	17 (LTS)
JavaFX	21.0.2
SQLite driver	sqlite-jdbc 3.45.1.0
Maven	3.8+
jpackage	Bundled with JDK 21
WiX Toolset	v3 (for the Windows installer)
Editor	VS Code

6. Design

6.1 Overall Architecture

The application is organized into four layers, each with a specific responsibility. This separation keeps the UI code from touching SQL directly and makes the codebase easier to reason about when something breaks.

The top layer is the user interface, built entirely in JavaFX. This includes the main window with its sidebar, the five module views, custom modal dialogs, and the toast notification system. The UI never runs SQL directly — it always goes through the data access layer.

Below that is the business logic layer, which holds logic that isn't tied to any single module. The `ConsistencyScore` class computes the daily score. `DateUtil` provides helpers like `today()`, `daysAgo(n)`, and `startOfMonth()`. `UndoManager` maintains a bounded stack of reversible actions for delete operations. `Toast` handles the ephemeral notification popups.

Next is the data access layer, consisting of five DAO classes — one per module plus a `DashboardDAO` for cross-module queries. Every SQL statement in the entire application lives in this layer. If the storage backend ever changes, only these files need updating.

At the bottom is the persistence layer — a single SQLite database file stored at `C:\Users<user>.anchor\anchor.db`. The `Database` class handles initialization, creates the schema on first launch, and exposes a shared connection to the rest of the app.

6.2 The Five Modules

Each module lives in its own JavaFX component class and has its own DAO. They share nothing except the database connection and the visual theme.

The Study Timer is a Pomodoro-style timer with configurable session and break lengths. When a session completes, it auto-logs to the `study_sessions` table with the date, start time, duration, and an optional subject tag.

The To-Do List handles task management. Tasks have a title, an optional due date, a priority level (Low, Medium, or High), and an optional category. Tasks due today or tomorrow show an orange highlight, and overdue tasks show red.

The Expense Tracker is a spending log. Each entry has an amount, a category, a date, and an optional note. The module shows a monthly total, a pie chart broken down by category, and a scrollable list of recent expenses.

The Habit Tracker manages recurring daily habits. Each habit has a name and can be ticked off once per day. The module calculates both the current streak and the longest streak for each habit, and shows a fire emoji when the streak is alive.

The Unified Dashboard is the aggregation view. It pulls data from all four modules using DashboardDAO and displays the consistency score, weekly charts for study time and spending, and a 60-day activity heatmap styled after GitHub's contribution graph.

6.3 Database Design

The most important design decision was to give every table a date field in ISO format, written as YYYY-MM-DD. This one shared field is what makes the unified dashboard possible. A single query can pull data from any combination of tables for a specific day, and the results combine without any date translation, timezone conversion, or formatting differences. Without this, the dashboard would need per-module date normalization logic, which would be fragile and error-prone.

There are five tables in total.

The study_sessions table stores each completed study session. It has an autoincrementing id, a date, a start_time in HH:MM format, a duration_minutes integer, and an optional subject string for what was studied.

The tasks table stores to-do items. It has an id, a title, an optional due_date, a priority (Low, Medium, or High), an optional category, an is_complete flag (0 or 1), and a created_date.

The expenses table stores individual spending entries. It has an id, a date, an amount stored as a real number, a category (Food, Transport, Books, Subscriptions, or Other), and an optional note.

The habits table stores habit definitions. It has an id, a name (which is unique), and a created_date.

The habit_logs table stores daily completion records. It has an id, a habit_id that references the habits table, a date, and an is_done flag. The table has a unique constraint on the combination of habit_id and date, which means each habit can only have one log entry per day.

That unique constraint was important for simplifying the toggle logic. When the user ticks a habit, the insert uses an ON CONFLICT DO UPDATE clause. Re-ticking the same day updates the existing row instead of failing or creating a duplicate. This made the UI code much simpler — I didn't need to query first to check whether a row existed before inserting.

6.4 Consistency Score Design

The consistency score is the centerpiece of the dashboard. Every day gets a score from 0 to 100, computed by combining activity from all four modules.

Each module contributes a weighted percentage. Study and habits each contribute 30 percent, tasks contributes 20 percent, and spending contributes 20 percent. The weights reflect what I think matters most for a student's day. Study and habits get the higher weight because they're the two long-term investments — you get better at studying by doing it daily, and habits compound over time. Tasks and spending are important but less central to daily consistency.

Each percentage is capped at 100 percent and calculated against a target. For study, the target is 60 minutes — anything at or above that gets full credit. For tasks, the target is three completed tasks. For spending, the target is staying under ₹500 — at or below that, the module contributes its full 20 percent, and above it the contribution decreases linearly so that spending ₹1000 contributes nothing. For habits, the target is completing all defined habits — if no habits are defined, this module contributes nothing, which is a gentle nudge to define at least one.

The formula is intentionally simple. Fancier approaches — weighting tasks by priority, or normalizing by day of week, or using a moving average — would add complexity without much practical benefit at this scale. A score that's easy to understand is a score I'll actually check. If I can't glance at the number and know why it is what it is, the metric loses its value.

6.5 Interface Design

The interface follows the Apple Human Interface Guidelines, adapted for a Windows desktop environment. The goal was something calm and modern rather than a traditional data-entry form.

The layout uses a fixed-width sidebar on the left with the five module names stacked vertically. The currently active module has a blue accent bar on its left edge and a slightly brighter background. The main content area on the right displays the selected module. The window starts maximized to take advantage of the full screen, but the user can resize or restore it at any time.

The color palette is dark with muted accents. The background is a near-black, cards are a slightly lighter gray, and text is off-white. Blue is the primary accent, used for buttons, active states, and the study chart. Green, orange, and red are used semantically — green for positive indicators like completed tasks and habit streaks, orange for due-soon warnings, and red for overdue tasks and destructive actions.

Typography uses the system's default sans-serif font at a small base size, with clear hierarchy. Section headings are larger and bolder, supporting text is smaller and more muted, and interactive elements use a medium weight to stand out without shouting.

The design deliberately avoids clutter. Each module fits on one screen without scrolling when the window is maximized, and the dashboard is arranged so the most important information —

the consistency score and today's stat cards — sits at the top, with the weekly charts and the 60-day heatmap below.

7. Implementation Tools and Technologies

Layer	Choice
Language	Java 17
UI	JavaFX 21.0.2
Charts	JavaFX Charts (BarChart, LineChart, PieChart)
Database	SQLite via sqlite-jdbc
Build	Maven + maven-shade-plugin
Packaging	JDK's jpackage
Installer	WiX Toolset v3

Things I Learned the Hard Way

The JavaFX launcher problem. This was the single biggest headache of the entire project, and I want to document it because I'm sure others will hit it too.

Everything worked fine when I ran the app from my IDE or with `mvn javafx:run`. But the moment I packaged it as a JAR with `jpackage`, it would install successfully but refuse to open. No error, no crash dialog, just... nothing.

After a lot of trial and error, I ran the app with a console window attached (`--win-console` on `jpackage`) and finally saw the actual error: text

Error: JavaFX runtime components are missing, and are required to run this application

The issue is that when a shaded (fat) JAR's main class extends `javafx.application.Application`, the JVM runs a module check at startup. Since the fat JAR has no module metadata, the check fails.

The fix is a small but essential trick: create a plain Launcher class that doesn't extend `Application`:

```
java public class
Launcher {

    public static void main(String[] args) {

        Main.main(args);

    }

}
```

Then tell `jpacakge` to use `com.studentdashboard.Launcher` as the main class instead of `com.studentdashboard.Main`. The JVM skips the module check because the entry point isn't an `Application` subclass, and everything works.

This took me several hours to figure out, mostly because the error was invisible without a console attached. Lesson learned: always test the packaged app with `--win-console` first.

The freeze on undo. Another issue that took me a while: when I added an Undo button to delete-confirmation toasts, clicking Undo would freeze the UI. The problem was that my toast was an overlay on the same `StackPane` as the list content. When Undo ran, it triggered `refresh()`, which rebuilt the list, which destroyed the toast's parent — while the toast's Undo button still had focus. JavaFX's focus traversal went into a loop.

The fix was twofold: (1) move the toast to a separate Popup window (so it's not inside the list's scene graph), and (2) dismiss the toast *before* running the undo action, deferring the actual work to the next UI pulse with `Platform.runLater`.

Custom dialogs. The built-in JavaFX Alert doesn't theme cleanly with custom CSS — its native title bar and icon look out of place next to a dark-themed app. I ended up building a custom `AppleAlert` class from scratch using a transparent `Stage`, which gives full control over appearance. More code, but a much better result.

The Build Pipeline

The complete build sequence: `bash`

```
mvn clean package mkdir staging copy
```

```
target\anchor-1.0.jar staging\  
jpackage --type exe --name Anchor ^  
--input staging ^  
  
--main-jar anchor-1.0.jar ^  
--main-class com.studentdashboard.Launcher ^  
--app-version 1.0 --vendor "Student" ^  
--win-shortcut --win-menu ^  
--dest dist
```

The output is dist\Anchor-1.0.exe — a standalone installer that bundles its own Java runtime. Someone can download it, install it, and run it without ever installing Java.

8. Testing

I tested the app manually across all five modules, plus integration tests of the dashboard aggregation logic. Here are the key test cases:

#	Test	Expected	Result
1	Run a 25-minute timer session	Session logged on completion	Pass
2	Add a task due tomorrow	Shows "due soon" (orange)	Pass
3	Add a task due last week	Shows "overdue" (red)	Pass
4	Add an expense	Total and pie chart update	Pass
5	Create a habit and tick it	Streak = 1	Pass
6	Untick a habit	Streak = 0	Pass

7	Delete a task	Confirmation, then toast with Undo	Pass
#	Test	Expected	Result
8	Click Undo on toast	Task restored, no freeze	Pass
9	Full day of activity	Score matches formula	Pass
10	Heatmap over 60 days	Intensity scales with activity	Pass
11	Fresh install on Windows 10	Runs without Java installed	Pass
12	Restart the app	Data persists	Pass

A Note on the Undo Fix

Test case 8 is worth calling out. Early versions of the app **failed** this test — the UI froze when Undo was clicked. The fix (described in Section 7) involved moving the toast to a separate popup window and dismissing it before running the undo action. After the fix, the test passes reliably even with rapid delete-undo-delete sequences.

Known Limitations

A few things I deliberately left out of v1:

- The consistency score formula is fixed — no way to customize the weights from the UI
- No data export to CSV or PDF
- Notifications are in-app only, no system tray integration
- Windows-only (no macOS or Linux builds yet)

None of these are blockers for personal use, but they're on the list for future versions.

9. Screenshots

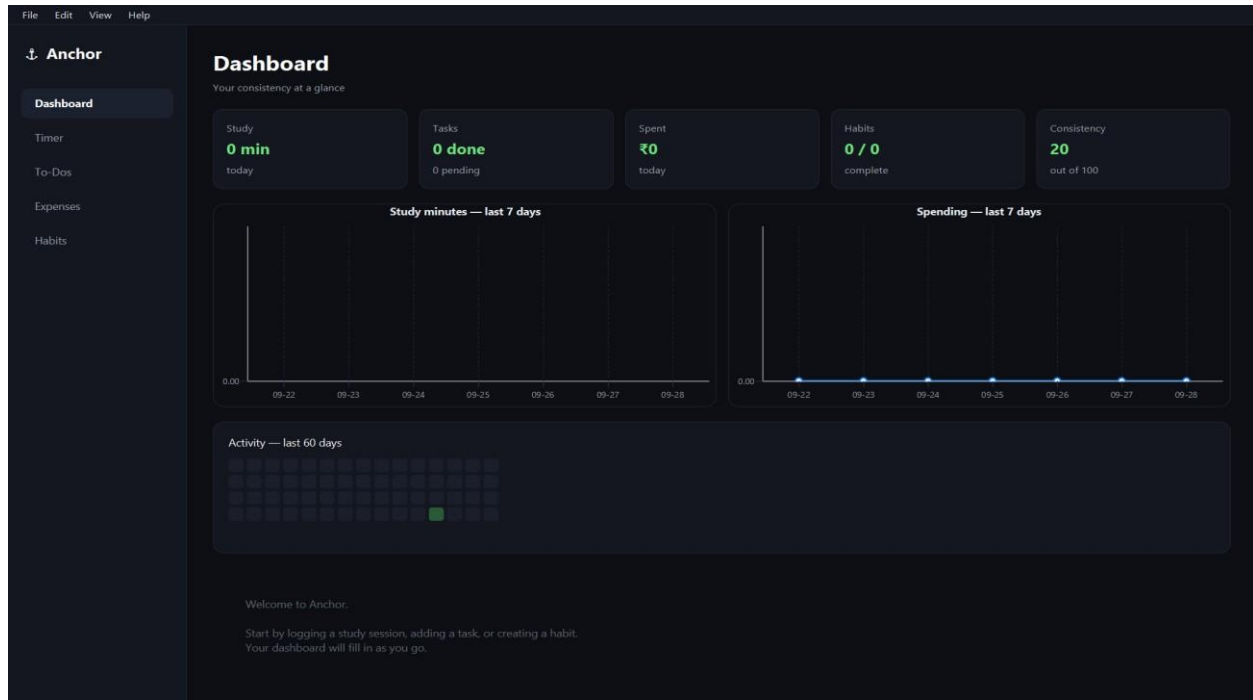


Figure 1: Dashboard — The unified view with consistency score, stat cards, weekly charts.

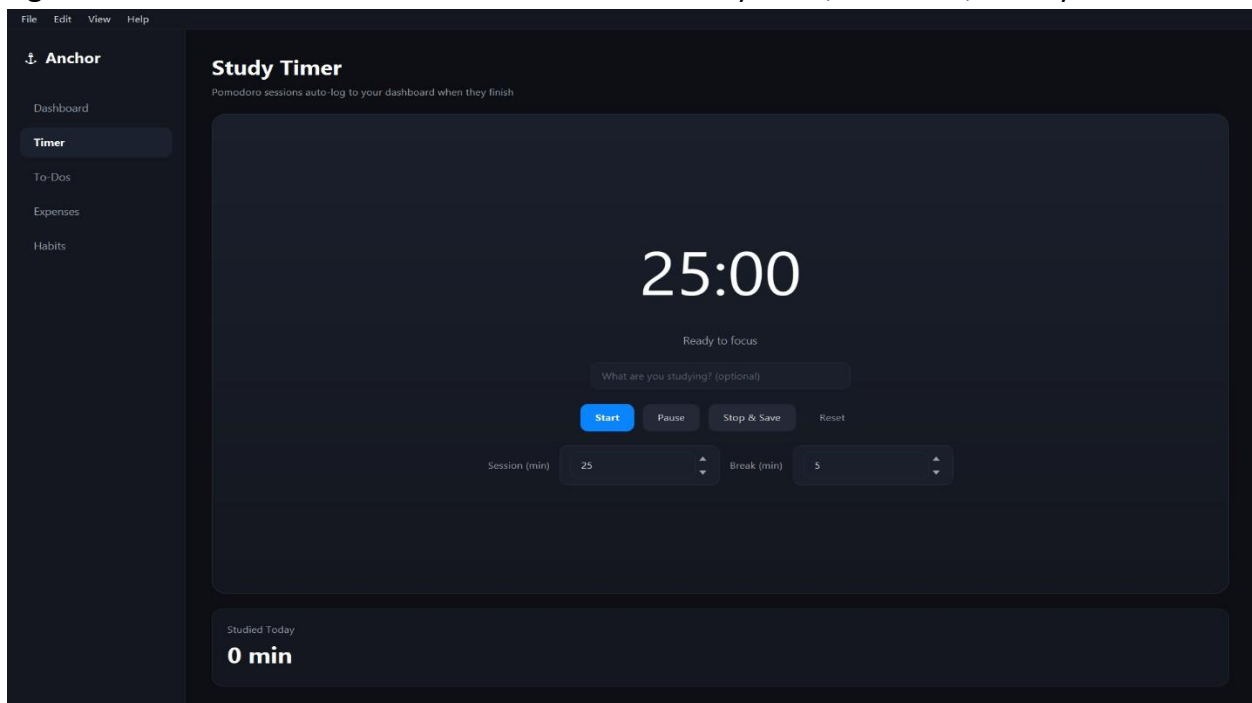


Figure 2: Study Timer — The Pomodoro timer with session controls and today's total study time.

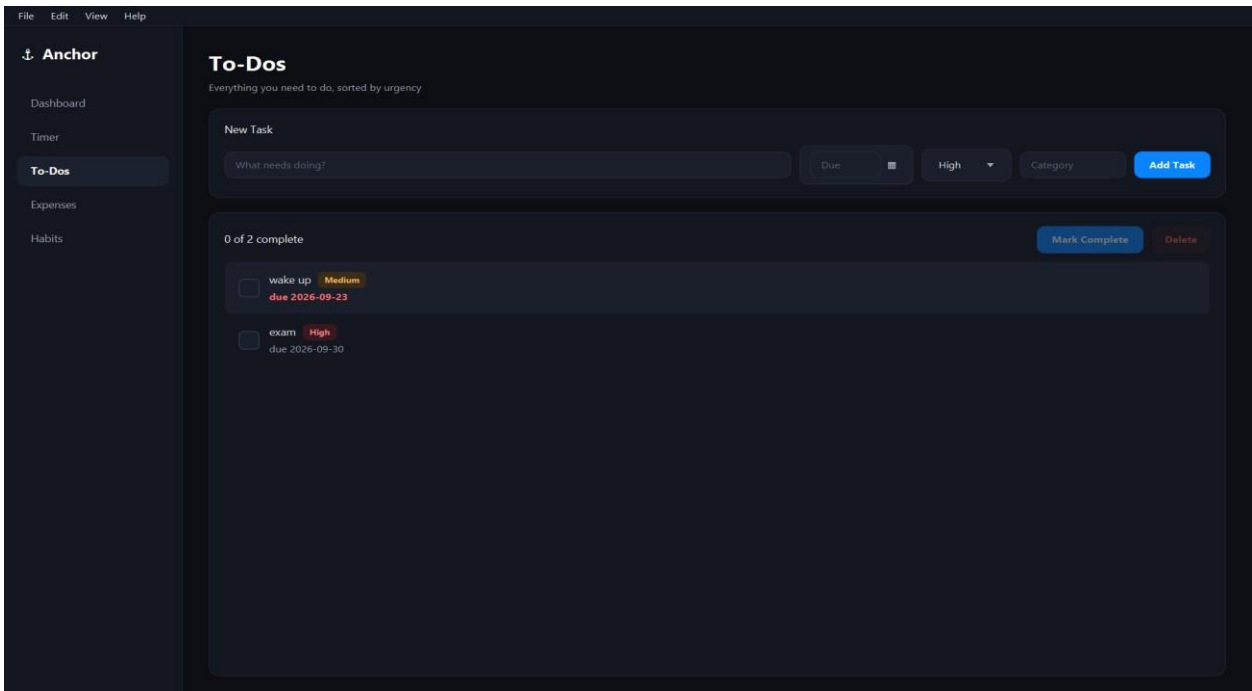
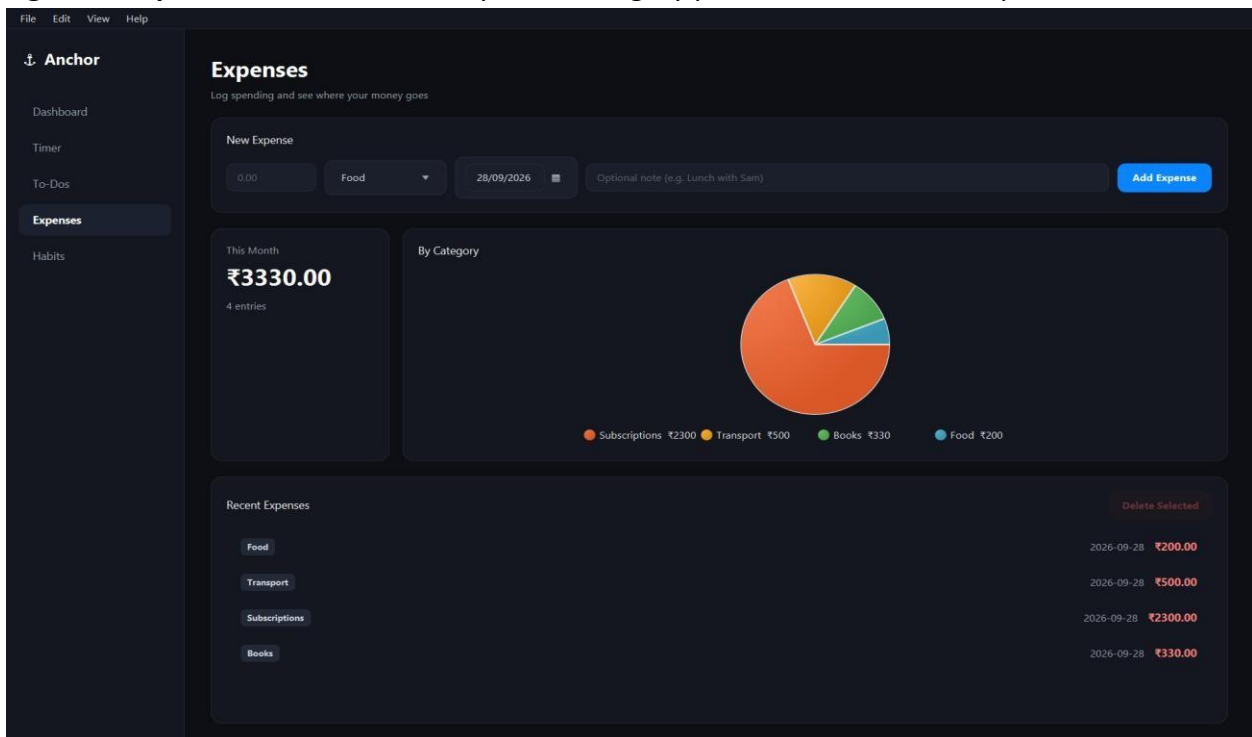


Figure 3: To-Do List — Task list showing priorities, due dates, and overdue highlighting.

Figure 4: Expense Tracker — Monthly total, category pie chart, and recent expenses



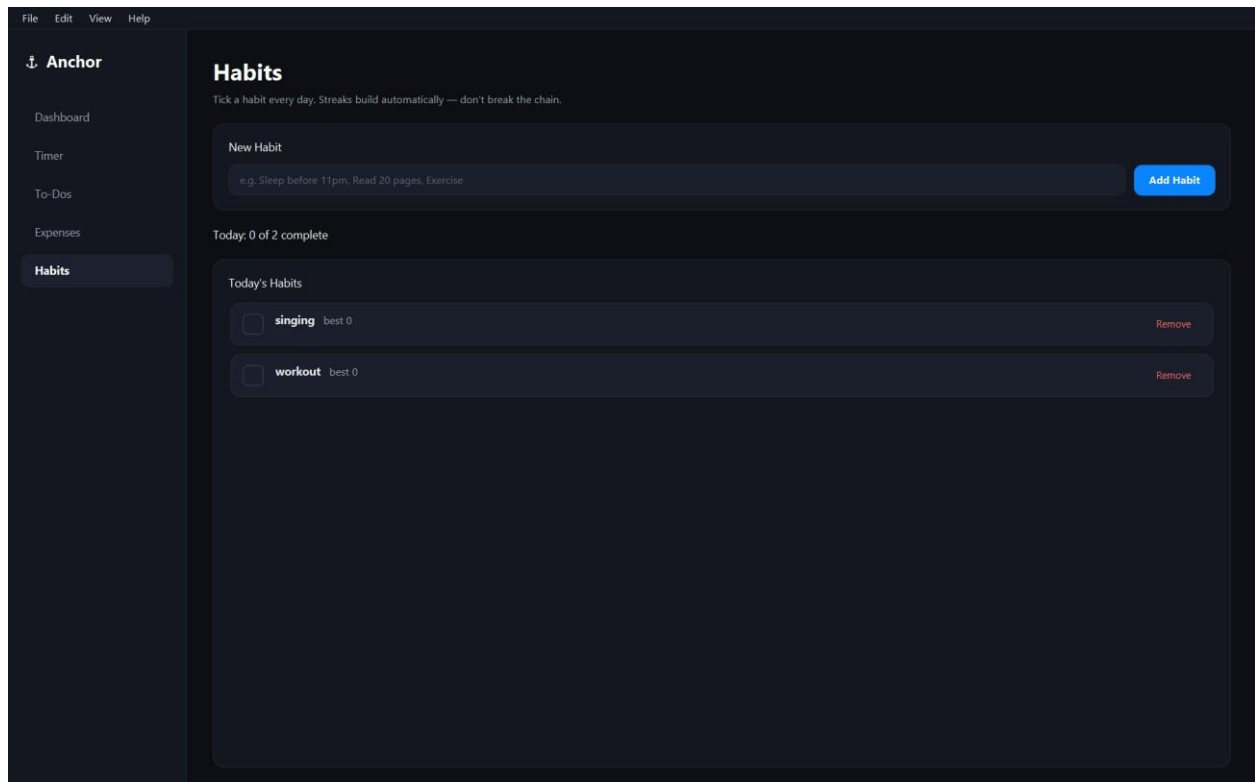


Figure 5: Habit Tracker — Active habits with current streak indicators.

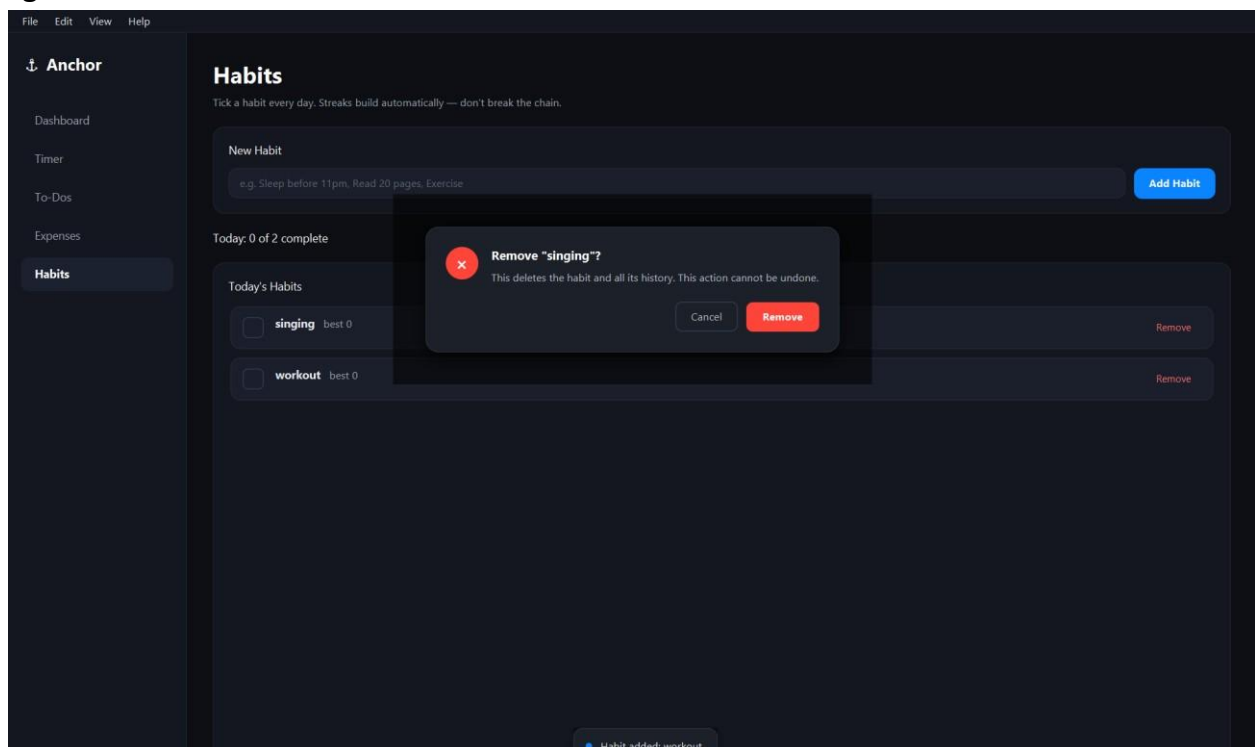


Figure 6: Custom Delete Dialog — The Apple HIG-inspired modal that confirms destructive actions.

10. Conclusion

Anchor does what I set out to build: it replaces four separate productivity apps with one, shares data across all four modules, and surfaces a unified consistency view on the dashboard. It runs as a native Windows application, stores everything locally, and installs cleanly from a single .exe.

The modules themselves came together fairly quickly — each one is straightforward UI plus a small DAO layer. The harder parts were in the details: getting the packaged app to launch at all, fixing the undo freeze, and building the custom dialogs to match the dark theme. Those took more time than all four modules combined.

If I were starting over, I'd do a few things differently. First, I'd set up the packaging pipeline earlier — right after the first module — so build issues surface before the codebase gets big. Second, I'd add the launcher class from day one instead of discovering it the hard way. Third, I'd write a few integration tests around the dashboard queries early, since those are what tie everything together.

But overall, I'm happy with where it landed. The app is stable, fast, and something I'll actually use daily. That was the whole point.

11. Future Work

Things I want to add in future versions:

1. **Data export** — CSV and PDF reports, especially for the weekly and monthly views
2. **System tray notifications** — reminders that don't require the app to be open
3. **Customizable consistency weights** — let users tune what matters most to them
4. **Cloud backup** — optional and end-to-end encrypted, for people who want it
5. **Cross-platform builds** — the same jpackage pipeline can produce macOS and Linux builds with minor changes
6. **Focus mode** — a full-screen timer view that hides everything else
7. **AI study assistant** — summarize notes, generate practice quizzes (much more ambitious, probably v2)
8. **Weekly review prompt** — a Sunday evening nudge to look back at the week

12. References

1. Oracle. *JavaFX Documentation*. <https://openjfx.io/>
 2. Oracle. *Java Platform, Standard Edition Documentation*. <https://docs.oracle.com/en/java/>
 3. SQLite Consortium. *SQLite Documentation*. <https://www.sqlite.org/docs.html>
 4. Xerial. *sqlite-jdbc*. <https://github.com/xerial/sqlite-jdbc>
 5. Oracle. *jpackage Tool Documentation*.
<https://docs.oracle.com/en/java/javase/21/jpackage/>
 6. Apple Inc. *Human Interface Guidelines*. <https://developer.apple.com/design/human-interface-guidelines>
 7. Apache Software Foundation. *Maven — Getting Started*.
<https://maven.apache.org/guides/getting-started/>
 8. Cirillo, F. *The Pomodoro Technique*. 2006.
 9. Duhigg, C. *The Power of Habit*. Random House, 2012.
-